

MODULE 07

React Hooks & Context in TypeScript

Hooks are the backbone of modern React. This module covers how to type `useReducer`, `useContext`, `useMemo`, `useCallback`, and custom hooks — the patterns that define a production React TS codebase.

~60 min read

Builds on Module 6

Production patterns

1. `useReducer` — Typed State Machines

`useReducer` pairs perfectly with discriminated unions from Module 4. The action union ensures you can only dispatch valid, known action types — no typos, no missing payloads:

useReducer with discriminated union actions

```
import { useReducer } from 'react';

// 1. Define your state shape
interface CounterState {
  count: number;
  step: number;
}

// 2. Define actions as a discriminated union
type CounterAction =
  | { type: 'increment' }
  | { type: 'decrement' }
  | { type: 'reset' }
  | { type: 'setStep'; payload: number }; // payload only where needed

// 3. Write the reducer — TS enforces exhaustive action handling
function counterReducer(state: CounterState, action: CounterAction): CounterState {
  switch (action.type) {
    case 'increment': return { ...state, count: state.count + state.step };
    case 'decrement': return { ...state, count: state.count - state.step };
  }
}
```

```
    case 'reset':    return { ...state, count: 0 };
    case 'setStep':  return { ...state, step: action.payload }; // TS
    knows payload exists here
  }
}

// 4. Use in component
function Counter() {
  const [state, dispatch] = useReducer(counterReducer, { count: 0, step: 1 });

  // dispatch is fully typed – TS knows exactly what's valid
  dispatch({ type: 'increment' }); // OK
  dispatch({ type: 'setStep', payload: 5 }); // OK
  dispatch({ type: 'setStep' }); // Error: payload missing
  dispatch({ type: 'fly' }); // Error: not a valid
  action
}
```

2. useContext — Fully Typed Context

Typing React Context correctly is one of the trickiest parts of React TS. The key challenge: `createContext()` needs an initial value, but often you don't have one at the top level. Here are two solid patterns:

Pattern 1 — Context with a default value (simplest)

Context Pattern 1 — default value

```
import { createContext, useContext, useState } from 'react';

// Define the context shape
interface ThemeContextType {
  theme: 'light' | 'dark';
  toggleTheme: () => void;
}

// Create context with a default value
const ThemeContext = createContext<ThemeContextType>({
  theme: 'light',
  toggleTheme: () => {}, // no-op default
});

// Provider component
```

```
export function ThemeProvider({ children }: { children: React.ReactNode })
{
  const [theme, setTheme] = useState<'light' | 'dark'>('light');
  const toggleTheme = () => setTheme(t => t === 'light' ? 'dark' :
'light');
  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

// Custom hook – the clean way to consume context
export function useTheme(): ThemeContextType {
  return useContext(ThemeContext);
}

// Usage in any component
function Header() {
  const { theme, toggleTheme } = useTheme();
  return <button onClick={toggleTheme}>Mode: {theme}</button>;
}
```

Pattern 2 — Context without a default (safer, throws if used outside Provider)

Context Pattern 2 — null guard with custom hook

```
interface AuthContextType {
  user: User | null;
  login: (email: string, password: string) => Promise<void>;
  logout: () => void;
}

// Pass null as default – tells TS context might not be provided
const AuthContext = createContext<AuthContextType | null>(null);

// Custom hook with null guard – throws a clear error if misused
export function useAuth(): AuthContextType {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used inside <AuthProvider>');
  }
  return context; // TS knows: context is AuthContextType (not null)
}

// Anywhere in the app:
function Profile() {
```

```
const { user, logout } = useAuth(); // fully typed, never null
return <div>{user?.name} <button onClick={logout}>Logout</button></div>;
}
```

Always wrap context in a custom hook

Instead of calling `useContext(MyContext)` directly in components, always create a `useMyContext()` wrapper. It keeps the null guard in one place, gives better error messages, and makes the import cleaner — components import `useAuth()`, not `AuthContext`.

3. `useMemo` — TypeScript Infers the Type

Good news: `useMemo` requires almost no extra typing. TypeScript infers the return type automatically from the computation function:

`useMemo` — mostly automatic

```
import { useMemo } from 'react';

// TS infers memoizedValue as number[]
const expensiveResult = useMemo(() => {
  return items.filter(i => i.active).map(i => i.value); // number[]
}, [items]);

// TS infers as { id: number; label: string }[]
const sortedUsers = useMemo(() => {
  return [...users].sort((a, b) => a.name.localeCompare(b.name));
}, [users]);

// Explicit type only needed if inference is ambiguous
const lookup = useMemo<Record<string, User>>(() => {
  return users.reduce((acc, u) => ({ ...acc, [u.id]: u }), {});
}, [users]);
```

4. `useCallback` — Memoized Typed Functions

Like `useMemo`, TypeScript infers the callback's type automatically. You mainly need to type the parameters when they can't be inferred from context:

useCallback — parameter types

```
import { useCallback } from 'react';

// TS infers handleClick as (id: number) => void
const handleClick = useCallback((id: number) => {
  dispatch({ type: 'selectItem', payload: id });
}, [dispatch]);

// Callback passed as prop — types must match prop interface
interface ListProps {
  onSelect: (id: number) => void;
}

function ParentComponent() {
  const handleSelect = useCallback((id: number): void => {
    console.log('selected:', id);
  }, []);

  // TS verifies handleSelect matches ListProps.onSelect
  return <List onSelect={handleSelect} />; // OK
}
```

5. Custom Hooks — The Full Pattern

Custom hooks are where TypeScript really shines in React. A well-typed custom hook is self-documenting — callers know exactly what they get back and what they need to provide:

A Typed useFetch Hook

useFetch<T> custom hook

```
import { useState, useEffect } from 'react';

// The hook's return type — define explicitly for clarity
interface UseFetchReturn<T> {
  data: T | null;
  loading: boolean;
  error: string | null;
  refetch: () => void;
}

// Generic hook — T is the data shape caller decides
function useFetch<T>(url: string): UseFetchReturn<T> {
  const [data, setData] = useState<T | null>(null);
```

```
const [loading, setLoading] = useState(true);
const [error, setError] = useState<string | null>(null);
const [tick, setTick] = useState(0);

useEffect(() => {
  setLoading(true);
  fetch(url)
    .then(r => r.json())
    .then((d: T) => { setData(d); setError(null); })
    .catch(e => setError(String(e)))
    .finally(() => setLoading(false));
}, [url, tick]);

const refetch = () => setTick(t => t + 1);
return { data, loading, error, refetch };
}

// Usage – caller specifies T, gets fully typed result
interface User { id: number; name: string; }

function UserProfile({ id }: { id: number }) {
  const { data: user, loading, error } = useFetch<User>(`/api/users/${id}`);
  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;
  return <h1>{user?.name}</h1>; // user is User | null here
}
```

A Typed `useLocalStorage` Hook

```
useLocalStorage<T> custom hook
import { useState, useEffect } from 'react';

function useLocalStorage<T>(key: string, initialValue: T): [T, (val: T) => void] {
  const [stored, setStored] = useState<T>(() => {
    try {
      const item = localStorage.getItem(key);
      return item ? (JSON.parse(item) as T) : initialValue;
    } catch { return initialValue; }
  });

  const setValue = (value: T): void => {
    setStored(value);
    localStorage.setItem(key, JSON.stringify(value));
  };
}
```

```
};

return [stored, setValue];
}

// Usage — T is inferred from the initial value
const [theme, setTheme] = useLocalStorage('theme', 'light'); //
string
const [counter, setCounter] = useLocalStorage('counter', 0); //
number
const [user, setUser] = useLocalStorage<User | null>('user', null);
```

6. forwardRef — Typing Forwarded Refs

When building reusable input components that need to expose a ref to the parent, use `React.forwardRef`. It takes two type parameters: the ref element type and the props type:

`forwardRef` with TypeScript

```
import { forwardRef, InputHTMLAttributes } from 'react';

interface InputProps extends InputHTMLAttributes<HTMLInputElement> {
  label: string;
  error?: string;
}

//           ref type           props type
//           |                   |
const Input = forwardRef<HTMLInputElement, InputProps>(
  ({ label, error, ...rest }, ref) => (
    <div>
      <label>{label}</label>
      <input ref={ref} {...rest} />
      {error && <span className='error'>{error}</span>}
    </div>
  )
);

// Usage in parent
function LoginForm() {
  const emailRef = useRef<HTMLInputElement>(null);
  return (
    <Input
      ref={emailRef} // TS verifies ref matches HTMLInputElement
```

```
        label='Email'  
        type='email'  
      />  
    );  
  }
```

7. The Power Pattern — useReducer + Context

Combining useReducer and Context gives you a lightweight, fully typed global state manager. This is the pattern behind many real-world React apps that don't use Redux:

```
useReducer + Context — full typed store  
// store/cart.tsx — fully typed cart store  
  
interface CartItem { id: number; name: string; qty: number; price:  
number; }  
interface CartState { items: CartItem[]; total: number; }  
  
type CartAction =  
  | { type: 'add';    item: CartItem }  
  | { type: 'remove'; id: number }  
  | { type: 'clear' };  
  
const initialState: CartState = { items: [], total: 0 };  
  
function cartReducer(state: CartState, action: CartAction): CartState {  
  switch (action.type) {  
    case 'add':  
      const items = [...state.items, action.item];  
      return { items, total: items.reduce((s, i) => s + i.price * i.qty,  
0) };  
    case 'remove':  
      const filtered = state.items.filter(i => i.id !== action.id);  
      return { items: filtered, total: filtered.reduce((s, i) => s +  
i.price * i.qty, 0) };  
    case 'clear': return initialState;  
  }  
}  
  
// Typed context  
interface CartContextType { state: CartState; dispatch:  
React.Dispatch<CartAction>; }  
const CartContext = createContext<CartContextType | null>(null);
```



```
export function CartProvider({ children }: { children: React.ReactNode })
{
  const [state, dispatch] = useReducer(cartReducer, initialState);
  return <CartContext.Provider value={{ state, dispatch }}>{children}</CartContext.Provider>;
}

export function useCart(): CartContextType {
  const ctx = useContext(CartContext);
  if (!ctx) throw new Error('useCart must be inside CartProvider');
  return ctx;
}

// In any component:
function CartButton({ item }: { item: CartItem }) {
  const { state, dispatch } = useCart();
  return (
    <button onClick={() => dispatch({ type: 'add', item })}>
      Add to cart ({state.items.length})
    </button>
  );
}
```

Module Summary

Everything you learned in this module:

- useReducer: `State` interface + discriminated union `Action` type = typed state machine
- Actions with payload: only add payload to the specific action types that need it
- Context Pattern 1: provide a default value — simplest, good for theme/locale
- Context Pattern 2: `createContext<T | null>(null)` + null guard in custom hook — best for auth/store
- Always wrap useContext in a custom hook — cleaner, better errors
- useMemo and useCallback: TypeScript infers the type automatically
- Custom hooks: return type as `interface UseFetchReturn<T>` — self-documenting API
- forwardRef: `forwardRef<HTMLInputElement, Props>(...` — two type params
- useReducer + Context = lightweight typed global state, no Redux needed

> Final Module: Module 8

Real-World React TS Patterns

API integration with typed fetch, typed forms, component composition patterns, generic components, and a complete typed React app architecture from file structure to deployment.